

Final Design Document

Introduction

Hello reader!

Our group chose the Watopoly game for our game. We put countless efforts into this project over these last two weeks, with many memorable all-nighters on campus! We truly hope you enjoy reading about our project, as we sure enjoyed making it!

Overview

We start from the main function, and then we come to the Controller class. This controller class is the class that we got inspiration from the MVC (Model View Controller) design pattern discussed in the lectures.

In order to describe the overall structure of our project, we will take you on a tour of our project. We start off like all other C++ programs in the main function. We designed our program so that we would spend as little time as possible in the main.cc file since we wanted to give input/output to a class specifically designed for it. In our case, this took the form of the Controller class.

The entrance to our game is defined in the playMonopoly() public method of Controller. This Controller class has two primary tasks:

1. Handle input from the user and command output to the user
2. Manage player turn-taking and control flow

We aimed to include as little game logic into the Controller as possible and delegate it all to the Model. To achieve this feat, we used several wrapper structs we call Responses. The Responses' contents differ, but they all have something called a context. The context is a string generated by an ostream and gives the user real-time dynamic information about what is happening on the gameboard. The idea is that we can call methods on the player whose turn it is, and get a Response object returned, which contains all the necessary data for both the Controller to know how to proceed and the user who is playing the game.

Here is an example of these responses in action:

```

MoveResponse res = p.move(roll);
p.notifyView();
cout << td << endl;
cout << res.context << endl;
if (res.action == Action::BuyOrAuction) {
    while (true) {
        string choice;
        cin >> choice;
        if (choice == "buy") {
            ChoiceResponse cr = p.buy();
            cout << cr.context << endl;
            if (cr.is_valid) break;
            else continue;
        } else if (choice == "auction") {
            commenceAuction(p, current_player_id);
            break;
        }
        else if (command(choice, p)) {
            continue;
        }
        else if (choice == "next") {
            cout << p.getName() << ": Must resolve current action" << endl;
        }
        else {
            cout << p.getName() << ": Invalid command" << endl;
        }
    }
}

```

At the top, we have the MoveResponse, which returns a context and an action. The action tells the Controller what to do. For example, if the user lands on an owned property but has insufficient money to pay tuition, p.move() (where p is a Player object, which we will talk about with the Model) will return Action::CantPayTuition. Another example and the one shown on the left is if the player lands on an unowned but ownable building, p.move() will return the action BuyOrAuction. This tells the Controller that it needs to prompt the user for a choice. The MoveResponse also contains the previously mentioned context string,

which provides all the information the user needs to know about where they landed. Whether good or bad news, the context will be generated dynamically from the Model and shown to the user.

Another example is the ChoiceResponse object. In this case, it has a context and a bool is_valid field. In this case, the user control flow gets held in a while (true) loop until the action is finished, which in this case means the player has sufficient funds to buy the building or they give it up for auction. The is_valid field allows us to keep the control flow in this loop depending on the outcome of the p.buy() method, and the context tells the user if the buy succeeded or failed. This design using Responses allows us to keep the Controller as simple as possible, offloading logic to the Model and keeping the class cohesive.

Moving onto the Model, as you briefly saw, the Controller can interact with the logic through the Player class, representing the current state of one of the users playing the game. Included here are fields like the player's balance, their owned properties if they are in jail, etc. These Player objects are the primary way the Controller and Model communicate with each other. Having this primary connection through the player allows us to make natural methods such as player.move(), player.buy() and player.declareBankruptcy(). The only other connection to the Model via Controller is through the Board Class, which represents a container for all 40 locations on the board. We represent these locations with an abstract class named Square. We extensively used inheritance and polymorphism here to model the different types of properties on the board, allowing all of them to be interacted with through a pure virtual method square.actionOnLand(). We made it our goal to make

connections natural and easy to reason about in the Model, lest we be cursed by unnecessary verbose.

The last location on our journey is the View. At the center of our implementation for our View is the observer pattern. We have classes Player and Square as children of an abstract Subject class and two new classes TextDisplay and SquareDisplay as observers. Here the idea is that every Player object has the TextDisplay observing it, and every Square has a SquareDisplay observing it. This means whenever the physical representation of a Player or Square changes (IE, the player's token moves, or Square's number of improvements change), the game display will be able to model them accurately.

Overall the MVC design allows all of these distinct parts (logic, display, input/output) to work individually. This allowed for better group work distribution and components, such as the View, to be switched out easily for different display mediums.

Updated UML:

Our design changed massively from what we had on the due date of one submission.

The most recognizable change we had on the UML is that we added many more methods than we had in our original submission. On top of that, we also made some class relationship changes, including the composition, aggregation and inheritance class relationships.

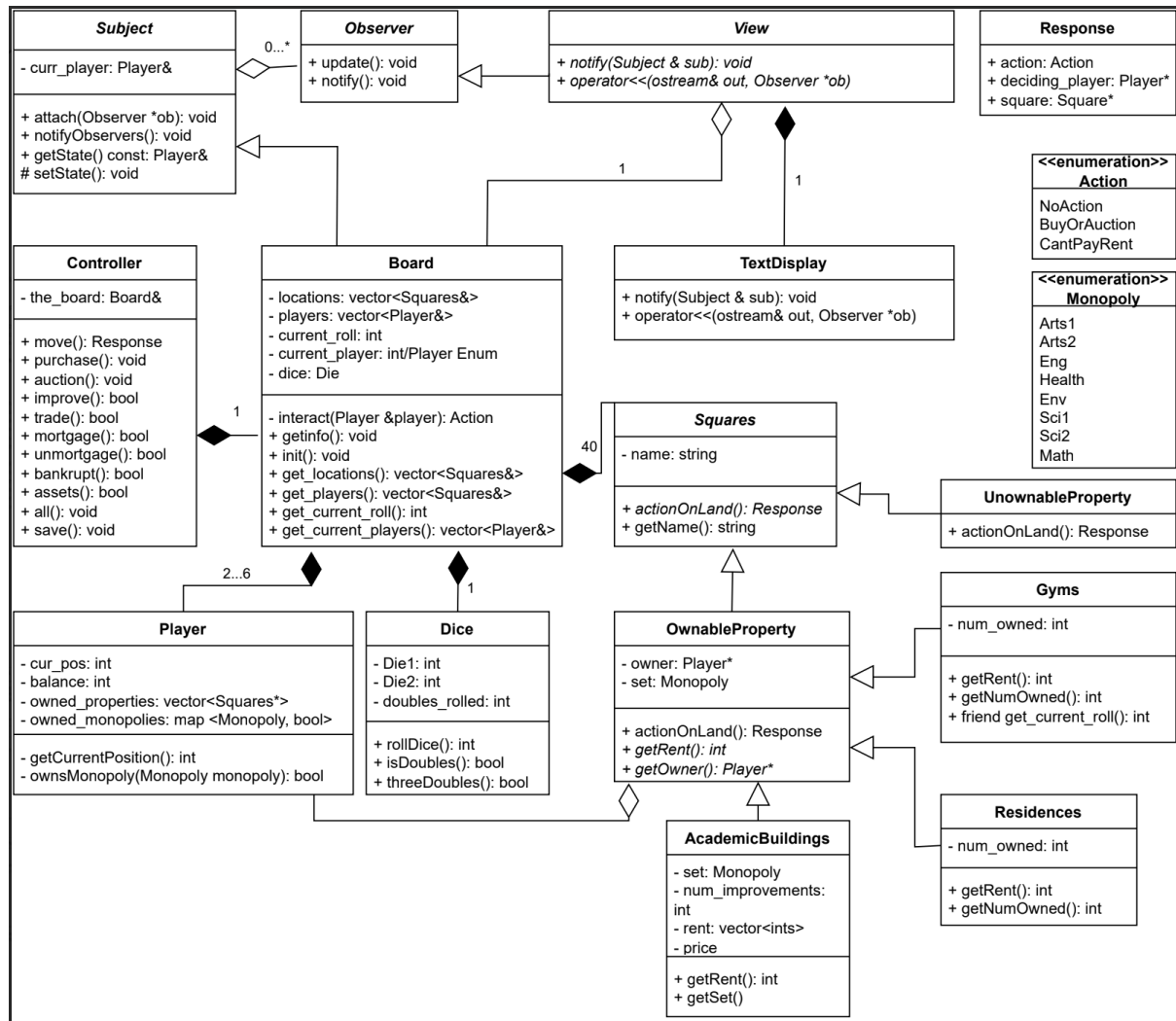
On a deeper level, our UML for Due Date 2 signified a shift in our design philosophy. Initially, we had everything centred around the gameboard, which would do most of the logic and control player turn flow. Early on, this created complicated, difficult-to-reason-about connections that were arbitrarily created from this desire to keep every and all logic away from the Controller.

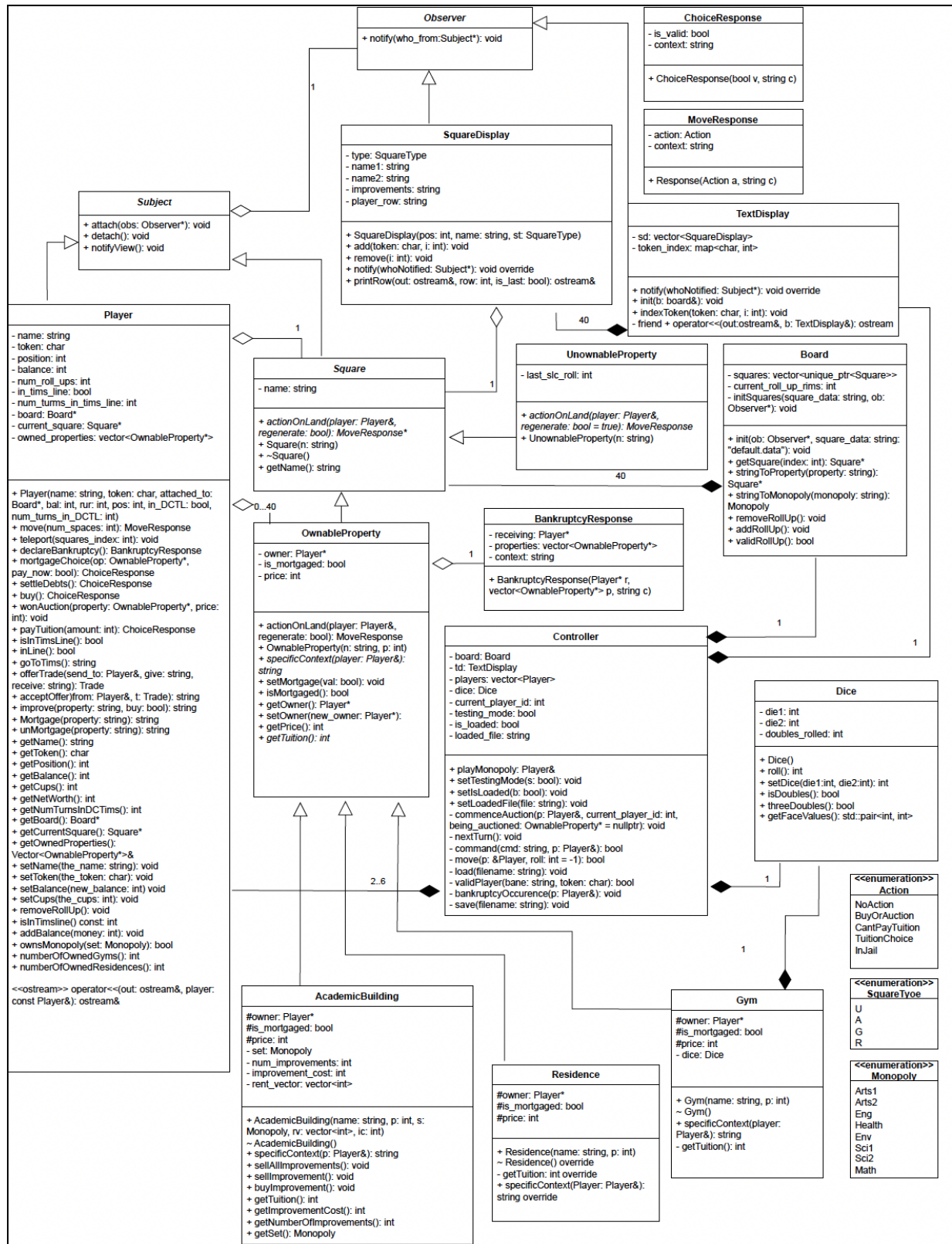
As we coded our game, we made a sacrifice on the cohesiveness of the Controller (input/output AND turn-taking) to allow for natural connections and a more resilient code base.

We also added many new classes, such as the SquareDisplay, and the different Response structs, which proved invaluable to our final design.

Just as a reference, we would like to include our previous UML and the new UML. The first one is the picture of the old UML, and the latter is the current (new) UML.

Previous UML:





Design:

We focused on using encapsulation and inheritance principles throughout our design. We used design patterns such as Model-View-Controller (MVC) pattern and the observer pattern. Inheritance was extremely beneficial since, throughout the squares, there is shared behaviour and a need to hold many objects in the same container.

Encapsulation allows the classes to keep their complicated logic hidden from their public implementations, allowing more efficient communication and interaction with other classes, and protecting against the dreaded spaghetti code.

Before using the Model-View-Controller pattern and observer, we encountered challenges in the project where there needed to be more consistency, difficulty in being able to test, no flexibility and tight coupling. Due to this, using this technique allowed us to separate components of the Watopoly game, where each component had its own responsibility. The model represents the logic; the view displays the board game to the user, and the controller handles user I/O and interacts with controls the game flow. The observer pattern then benefited from this choice of design technique because it allowed the view component to observe changes to the model and then update the display when state occurs a change/update.

Resilience to Change:

Before implementing, our goal was to focus on a design that would support modularity and flexibility, allowing changes in the program to be made without affecting other components. We found this a crucial topic that should be discussed during the initial UML stage. Due to this, our design supports the possibility of various changes to the program specification due to the high level of abstraction and encapsulation, a modular design (MVC), low coupling between modules and incorporating of the observer pattern. With abstraction and encapsulation, it allows and makes it easier for us, as developers, to be able to modify individual components without making an impact on the rest of the components/systems. We also have low coupling as a part of our design where the components and modules are connected, but if a change occurs in one, the other module wouldn't need to be aware.

This design allows us to add various and multiple changes with a process that encounters little to no bugs. Without using the Model-View-Controller (MVC) pattern in our design of the Watopoly game, we wouldn't be able to add significant changes as we would with the current design. Adding changes would increase the number of bugs and errors in other components of the codebase, where it could result in broken code. Spending multiple hours on debugging shows that the design would need change. Due to this, focusing on resilience to change early on has prevented us from going back in the code and re-implementing a new design approach later on.

Answers to Questions (the ones in the project specification):

This design allows us to add various and multiple changes with a process that encounters little to no bugs. Without using the Model-View-Controller (MVC) pattern in our design of the Watopoly game, we wouldn't be able to add significant changes as we would with the current design. Adding changes would increase the number of bugs and errors in other components of the codebase, which could result in broken code. Spending multiple hours on debugging shows that the design would need to change. Due to this, focusing on resilience to change early on has prevented us from going back into the code and re-implementing a new design approach later on.

- 1. After reading this subsection, would the Observer Pattern be a good pattern to use when implementing a gameboard? Why or why not?*

Before agreeing that the Observer Pattern would be a good pattern to use when implementing the gameboard, we initially had decided it would not be helpful for Watopoly; however, after deciding on using MVC (Model View Controller), we discovered the observer pattern would be an excellent pattern to use. The observer pattern has many benefits when working with the MVC architecture since it helps reduce coupling, where the changes can occur in the components, but they don't need to affect one another, making the code easier to modify and increasing flexibility.

The Observer Pattern helps with flexibility since it allows new views to be added without impacting the model. New views can include a console interface, a graphical UI or a mobile app interface. Not only this, but we also agree that using MVC and Observer Pattern separates the logic and UI (user interface). This separation increases flexibility and is more effective in not affecting the watopoly's game logic. This can help significantly when debugging, testing, and adding more changes/features to a specific component and not requiring the changes to be implemented or affecting other parts.

- 2. Suppose that we wanted to model SLC and Needles Hall more closely to Chance and Community Chest cards. Is there a suitable design pattern you could use? How would you use it?*

The Community Chest and Chance cards offer a challenge to implement in that each card has different behaviour, as opposed to the predictable outcomes from Watopoly's SLC and Needles Hall. The Chance and Community Chest cards would have to be hard-coded somewhere, but ideally, we do not want to hard-code it into our game board logic. I don't see a specific design pattern, at least from the ones we've seen in class, that would help with this problem. This would imply we may want to use a design pattern such as the [action/command pattern](#) (which was not covered in class). Using this, we could have Needles Hall own a "Needles Hall"

class, and SLC own an "SLC" class. Both of these classes would have a pure virtual method use(). The children of Needles Hall and SLC would be wrapper classes with an overridden use() function. This function, which would be called by its parent's use(), would execute the desired effect of the card. Each of these wrapper classes could be instantiated inside of an array in Needles Hall or SLC so that the gameboard can assign a NeedlesHall* or SLC* to a random index in this array (using <random>). Finally, the gameboard can call ->use() to get a random effect. The benefit is that to change the cards, such as if you had a different set of Chance cards or if you wanted fewer or more cards in play, you only have to change the NeedlesHall.cc or SLC.cc.

3. Is the Decorator Pattern a good pattern to use when implementing Improvements? Why or why not?

Initially, our team thought the decorator pattern would be a good idea to use for modelling improvements on the board. While adding complexity to the program, the decorator pattern shines because it allows features to be added dynamically to a class. Intuitively, this makes sense with improvements because instead of spending a tedious amount of time to hardcode the tuition price after each improvement, you could take the tuition from all previous improvements and the base property and add the tuition that the current improvement adds.

As we discovered, the issue with the decorator pattern was that finding the tuition the current improvement adds is not feasible. Initially, it seems like the tuition is multiplied by 3, but you soon realize this isn't the case. After doing some research, it turns out there is no pattern at all for the scaling of Monopoly improvements. If there is no pattern, then the tuition prices for each improvement must be hardcoded, meaning we can't find the rent price dynamically. This means all of the benefits of the decorator pattern are gone, but the added complexity is still present. For this reason, we decided not to use the decorator pattern.

Extra Credit Features:

Yes, we have implemented the extra credit features. The features that we have implemented for the extra credit are as follows:

First and foremost, we have implemented the four marks out of the ten available bonus marks, which is not to use any delete statements in the entire program.

For extra features, we primarily worked to make our game as rigid as possible, making it difficult for users to break with input and ensuring they have an enjoyable experience the whole way through.

We also made a special effort to display dynamic and helpful information to the user after every command. We achieved this using the Response structs mentioned in the structure overview section.

Final Questions:

1. What lessons did this project teach you about developing software in teams? If you worked alone, what lessons did you learn about writing large programs?

Developing software in teams requires a lot of push, pull and merging of the code in a timely manner. It also taught us that everyone has a unique way of approaching the same coding question, which really expands intellect, and it is only possible if working in teams. Working in teams increases a lot of growth mindset because it keeps everyone motivated to stick to the same task. While working in teams, we try to aim for everyone's benefit, not just individual benefit. We also learned the importance of communication within a team-oriented environment in software development. Since we all work on the same codebase, changes can occur at different times. These changes could include a new function that needs to be added or a new class that can impact the parts another team member can work on. Due to this, communicating our latest ideas and changes to one another allows everyone to be on the same page; this decreases confusion, and further increases the efficiency spent on implementing the software.

Moreover, it also helps to interpret the task well, i.e., if one of the group members needs to understand a concept better, then one of the other group members could help them understand it better. This allows our team to learn from one another and increase one's knowledge. Also, It develops a sense of urgency to complete the project by enabling us to see the importance of time management. Moreover, developing this project in a team was a fantastic learning experience where members with different technical backgrounds could combine their skills and knowledge into one masterpiece.

2. What would you have done differently if you had the chance to start over?

If we had the chance to start over, we could have started earlier and implemented more than the bonus features specified in the document. Since, while doing this, we had fun creating a monopoly game and thought that if deeply done, this is quite an exciting project and might even take months or more to implement with the fullest possible functionality. This means if we had the chance to start over, we would have thought more about additional features that could have been prioritized. However, we also believe in focusing on the current functionality and mastering should be focused on more. This is because having an efficient base and design will further allow changes to be made more easily!

Conclusion:

We learnt a lot about the Object Oriented Programming paradigm via this project. It was beneficial to work on this project within a team where we could work on creating Watopoly, a project we all were passionate about creating. We also found it nice to see how we combined all concepts and knowledge we learned from the course throughout the last four months. Being able to choose how to design, implement and make changes when needed was an experience that we needed in development. Our team would sincerely like to thank the CS course staff for allowing us to grow, learn and become better developers through this final project.